05 按操作系统运行代码

只看自己的系统，跟着做一次

这册是前面四个主题的配套操作手册，不额外增加第六天。Windows、macOS、Ubuntu / WSL 的源代码共用，安装方式、终端语法和部分进程行为分开说明。

目录与运行入口

```text
学习资料 / 配套示例 /
  macOS / README.md、run_examples.sh
  Windows / README.md、run_examples.ps1
  Ubuntu-WSL / README.md、run_examples.sh
  run_examples.py        共用启动器
  *.py、*.sv、*.scala    共用教学源代码
```

先找到“配套示例”，在终端进入该目录，再运行对应入口。启动脚本只执行例子，不会替你安装软件或修改系统配置。

系统	运行 Python 练习	安装工具的地方
macOS	bash macOS/run_examples.sh python	Terminal 终端
Windows	py -3 run_examples.py python	PowerShell
Ubuntu / WSL	bash Ubuntu-WSL/run_examples.sh python	Ubuntu 终端

末尾的 python 是练习组名称，不是文件名。换成 rtl 运行四组仿真；换成 chisel 生成 RTL。先完成 Python，再继续安装硬件工具。

先获得文件

如果从 GitHub 下载，本仓库当前是私有仓库，需要相应访问权限。已登录网页可选择 Code → Download ZIP；也可以在配置好 GitHub 身份验证后克隆。

解压 ZIP 的目录可能叫 Graduation-Project-main，克隆目录通常叫 Graduation-Project。下面的路径只是例子，请按自己的位置替换。不会定位文件夹时，优先使用系统说明中的拖拽或地址栏方法。

macOS ① 安装与定位文件夹

适用于 Apple Silicon 和 Intel Mac

用 Spotlight 搜索“终端”或 Terminal 并打开。先安装命令行开发工具；系统提示已安装时即可继续：

```bash
xcode-select --install
```

接着打开 Homebrew 官网，按首页说明安装，并完成安装器给出的 Next steps。官网见 [M1]；已有 Homebrew 可跳过此步骤。

装齐本周需要的工具

```bash
brew --version
brew install python icarus-verilog scala-cli
python3 --version
iverilog -V
scala-cli version
```

Homebrew 中包名为 icarus-verilog；安装后运行命令叫 iverilog 和 vvp。只做 Python 实验时，可先仅安装 python。Scala CLI 会管理启动器要求的 Java 21。[M2][M3]

进入代码目录

假设仓库在下载文件夹中：

```bash
cd "$HOME/Downloads/Graduation-Project/学习资料/配套示例"
pwd
ls
```

$HOME 表示自己的主目录；pwd 显示当前目录，ls 列出文件。如果解压目录有 -main 后缀，相应修改路径。也可先输入 cd ，把 Finder 中的“配套示例”文件夹拖进终端，再按回车。

找不到 brew 时

重新开终端，并执行安装器给出的路径配置。Apple Silicon 默认路径通常为 /opt/homebrew，Intel 通常为 /usr/local；按实际安装路径选择，不要把两个架构的工具混着用。详细命令见 macOS/README.md。

> 本次没有实体 Mac，macOS 命令依据官方文档编写与核对；后面的实际测试记录将 Windows / Ubuntu 与 macOS 分开标注。

macOS ② 运行全部教学例子

第一步：Python 的进程实验

在“配套示例”目录运行：

```bash
bash macOS/run_examples.sh python
```

启动器依次运行创建与等待、管道通信、终止与回收、策略模型。若想逐个学习，使用：

```bash
python3 process_demo.py
python3 pipe_demo.py
python3 terminate_demo.py
python3 policy_model.py
```

第二步：RTL 仿真

```bash
bash macOS/run_examples.sh rtl
```

会检查选择器、计数器、手写停止门及已有 Chisel 生成模块。输出应有相应 PASS；计数器波形位于 `outputs/counter4.vcd`。文本结果与逐拍表足以完成基础练习，波形查看器可选。

第三步：Chisel 构造与生成

```bash
bash macOS/run_examples.sh chisel
```

首次会下载固定依赖与后端。输出包含 `module SafetyGate` 等 SystemVerilog；这一步是生成，不等于 FPGA 实现或形式证明。

手工命令也能跑

```bash
iverilog -g2012 -s counter4_tb -o counter4.vvp counter4.sv counter4_tb.sv
vvp counter4.vvp
```

```bash
scala-cli run chisel_project.scala SafetyGate.scala \
  --main-class EmitSafetyGate --server=false --jvm 21
```

反斜杠放在行末表示 Bash 命令续行；PowerShell 不能照搬这种续行方式。也可把整个 Chisel 命令合成一行。

Windows：按 PowerShell 路径运行

Python 先行，硬件工具按需准备

先按 Python 官网安装 Python 3，并检查 `py -3 --version`。已有 python 命令、没有 py 时，可把下文 `py -3` 换成 python。

在文件管理器进入“配套示例”，在地址栏输入 powershell 并回车。也可以用作者的路径示例定位，路径应按自己电脑修改：

```powershell
Set-Location -LiteralPath 'D:\Projects\毕设\学习资料\配套示例'
py -3 run_examples.py python
```

若想逐个学习，执行 `py -3 process_demo.py` 等文件。目录里也有 Windows/run_examples.ps1；若脚本被执行策略拦截，直接运行上面的 Python 命令，无需修改全局策略。

RTL：原生 Icarus 或 WSL

已有原生 Icarus 时，先检查再运行：

```powershell
Get-Command iverilog,vvp
py -3 run_examples.py rtl
```

没有原生工具，可进入 WSL Ubuntu，再读下一页。PowerShell 与 Ubuntu 终端的安装命令需要分开，不能把 sudo apt 直接粘贴到 PowerShell。

Chisel：Scala CLI

WinGet 可用时安装，随后重新打开终端并回到代码目录：

```powershell
winget install virtuslab.scalacli
```

```powershell
scala-cli version
py -3 run_examples.py chisel
```

若原生 Windows 的生成后端不支持或失败，可使用 WSL；此前的 Python 实验不依赖该后端。Windows 逐步说明见配套 Windows/README.md。[M3][C2]

Ubuntu / WSL：在 Ubuntu 终端运行

Windows 用户先确认已经进入 WSL Ubuntu

原生 Ubuntu 与 WSL Ubuntu 在此使用相同命令。Windows 用户可先在 PowerShell 输入 wsl，再确认自己进入了 Ubuntu 环境。

```bash
sudo apt update
sudo apt install python3 iverilog
```

定位代码

WSL 中 D 盘路径示例：

```bash
cd '/mnt/d/Projects/毕设/学习资料/配套示例'
```

原生 Ubuntu 若克隆在主目录：

```bash
cd "$HOME/Graduation-Project/学习资料/配套示例"
```

两条路径选一条，并按自己的位置修改。接着运行：

```bash
bash Ubuntu-WSL/run_examples.sh python
bash Ubuntu-WSL/run_examples.sh rtl
```

Chisel 工具

按 Scala CLI 官方 Linux 安装步骤准备工具，已有安装可跳过。安装完成后执行安装器提示的 PATH 配置，重新打开终端，再回到代码目录。[M3]

```bash
scala-cli version
bash Ubuntu-WSL/run_examples.sh chisel
```

启动器使用 Java 21；首次运行需要下载依赖与后端。命令参数固定了教学依赖，但工具安装方式会随官方发布变化。

Linux 的进程列表与 cgroup 观察命令见第 03 册；这些专项接口需要在 Linux 环境中使用。

三系统结果怎样对照

观察行为，不要求每台机器打印相同编号

实验	应看到的共同现象	可能不同的部分
process_demo	父子创建关系、子程序退出、wait 返回	PID、部分输出顺序
pipe_demo	收到 hello from child	一般无需比较编号
terminate_demo	已知子进程结束并被等待	终止机制与退出码
policy_model	PASS，8 条断言通过	平台不改变其逻辑规则
RTL	所列定向测试通过	工具版本提示与日志格式

POSIX 平台上 Python terminate 发送 SIGTERM，kill 发送 SIGKILL。Windows 中 terminate 使用 TerminateProcess，kill 是它的别名。macOS / Ubuntu 的退出码可能为 -15；Windows 本次环境观察到 1。重点是理解其对应行为。[P1]

Mac 可以观察什么

```bash
ps -o pid,ppid,state,comm -p $$
```

也可打开“活动监视器”。这些帮助观察进程，但 Linux 的 cgroup、namespace 和 seccomp 属于另一套系统接口；需要实际操作时，再使用 Linux 虚拟机或远程主机。

RTL 入口第四项在检查谁

它检查仓库里已经提供的 SafetyGate.generated.sv。chisel 入口会打印本次生成结果，不会自动覆盖这个文件。重新比较时，应先按生成文件分隔标记提取主模块，再使用适配器和测试平台。

实际验证范围

此前的 Python 已在 Windows 与 Ubuntu 运行，RTL 与 Chisel 已在 Ubuntu / WSL 验证。本次还会检查新增启动器与脚本路径；macOS 尚无实体设备运行记录，不能把 Bash 在 Linux 上通过等同于 Mac 实测。

常见错误：先定位发生在哪一层

同一个错误，不要反复盲试安装

现象	优先检查	下一步
找不到文件或脚本	pwd / Get-Location 是否在配套示例	重新定位，不要改源码
找不到 python3 / py	Python 是否安装、命令名是否正确	按自己系统入口检查
Missing iverilog / vvp	仿真工具是否在 PATH	macOS 用正确的 Homebrew 包名
找不到 brew	安装器 Next steps 是否完成	按实际架构配置路径
scala-cli 下载失败	网络、仓库可达性与报错 URL	保留完整错误，核对官方文档
firtool 架构错误	系统架构与工具是否一致	Mac 避免混用 Rosetta 与原生工具
Mac 找不到 /proc	使用了 Linux 特定步骤	改看三系统对应关系
Windows 不允许运行 ps1	PowerShell 执行策略	直接用 py -3 run_examples.py

依赖仓库有问题时

只有确认下载故障来自 Maven 仓库选择时，可在 macOS / Ubuntu 对单次命令指定中央仓库：

```bash
COURSIER_REPOSITORIES=https://repo.maven.apache.org/maven2 \
  bash macOS/run_examples.sh chisel
```

Ubuntu 将路径换成 Ubuntu-WSL/run_examples.sh；Windows 原生不要照搬这种变量前缀语法。网络恢复前，可先读已有 RTL 并做逐拍推演。

若卡在 JDK 索引下载，且已有 Java 21，可用 chisel 组的 `--java-home` 参数指向已有 JDK；具体路径与命令见自己系统的 README。库和后端首次使用仍需联网。

文件名和路径

路径有空格、中文时要保留引号。脚本根据自身位置寻找共享文件，所以整个“配套示例”文件夹应一起保留。不要只拷贝 macOS 或 Windows 子目录。

学习时间仍按五天安排。遇到环境问题先做可运行的 Python 与手工推演，并记录报错，避免把全部时间花在安装上。

检查清单与官方入口

在自己的系统打勾

- 我知道当前终端属于 Windows、macOS 还是 Ubuntu。
- 我能定位“配套示例”，并找到共享的源代码文件。
- 我运行了 Python 组，能解释父子进程与等待回收。
- 我运行了 RTL 组或完成逐拍推演，能解释复位与保持。
- 我知道 Chisel 生成与 RTL 仿真是两个阶段。
- 我知道哪些第 03 册内容属于 Linux 专项。
- 我记录了自己的系统、架构、工具版本和实际结果。

运行记录模板

```text
系统 / 版本 / 架构：
终端类型：
Python / Icarus / Scala CLI 版本：
运行的命令：
输出或报错：
我能解释的现象：
还需要核实的问题：
```

官方参考

[M1] Homebrew：https://brew.sh/

[M2] macOS Icarus：https://formulae.brew.sh/formula/icarus-verilog

[M3] Scala CLI：https://scala-cli.virtuslab.org/install/

[C2] Chisel：https://www.chisel-lang.org/docs/installation

[P1] Python subprocess：https://docs.python.org/3/library/subprocess.html

[M4] Homebrew 路径与安装：https://docs.brew.sh/Installation

更完整的分系统说明位于配套示例中的三个系统目录；验证范围与教学边界见“示例说明与验证记录”。